

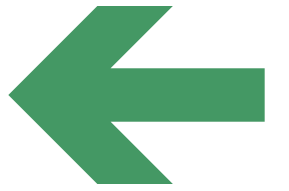
SQL Project Report

Online Food Delivery

Advance Level



Kiran Kavala
kavalakiran98@gmail.com



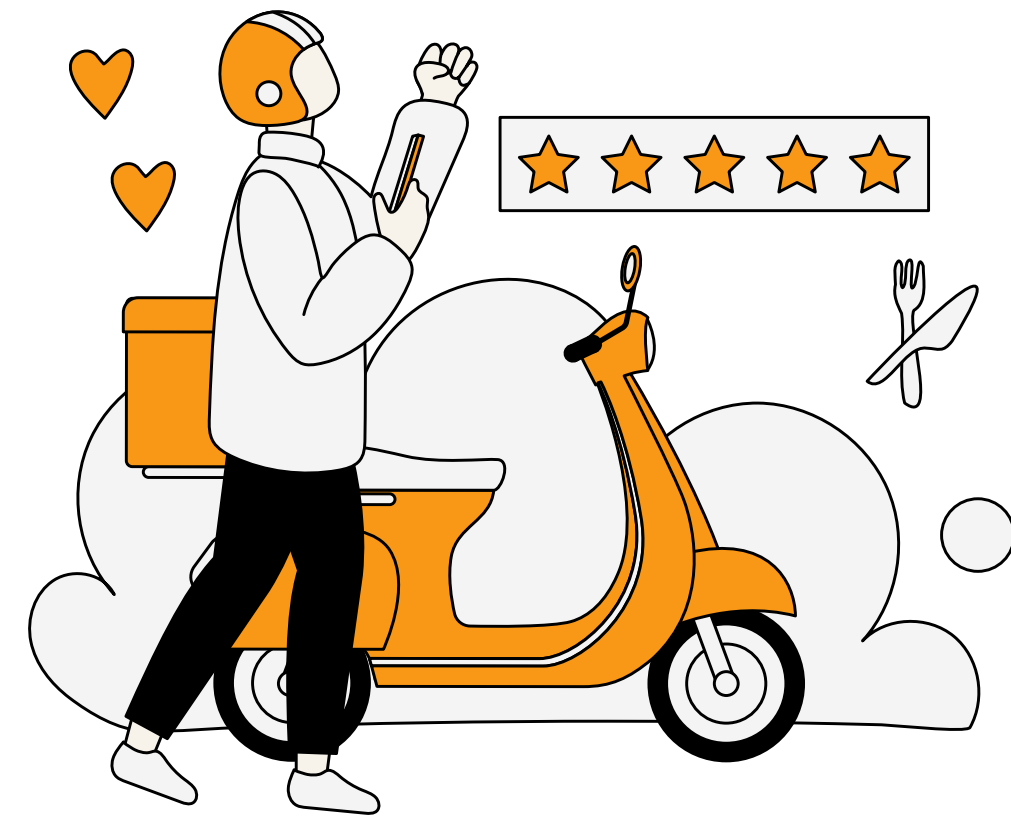
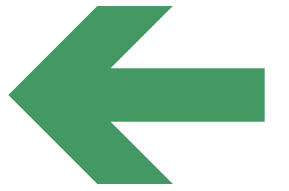
What the project is about

The Online Food Delivery SQL Project analyzes customer ordering behavior, restaurant performance, menu pricing, and sales trends using MySQL. The project demonstrates advanced SQL concepts such as CASE statements, Common Table Expressions (CTEs), Window Functions, Temporary Tables, Views, Aggregate Functions, and Subqueries to solve real-world business problems. The objective is to extract meaningful insights that help restaurants improve customer experience, optimize menu offerings, and increase overall revenue.



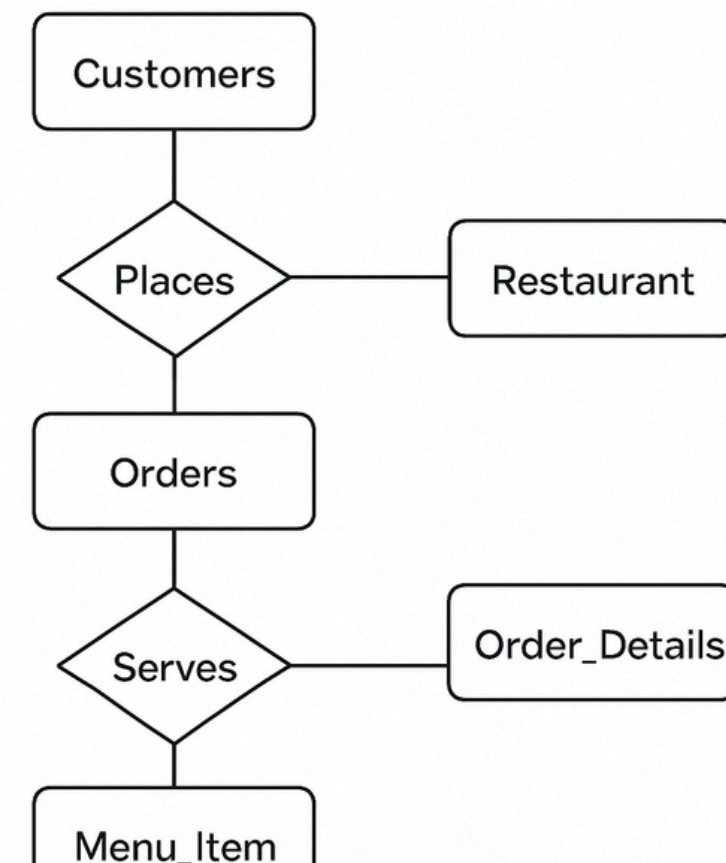
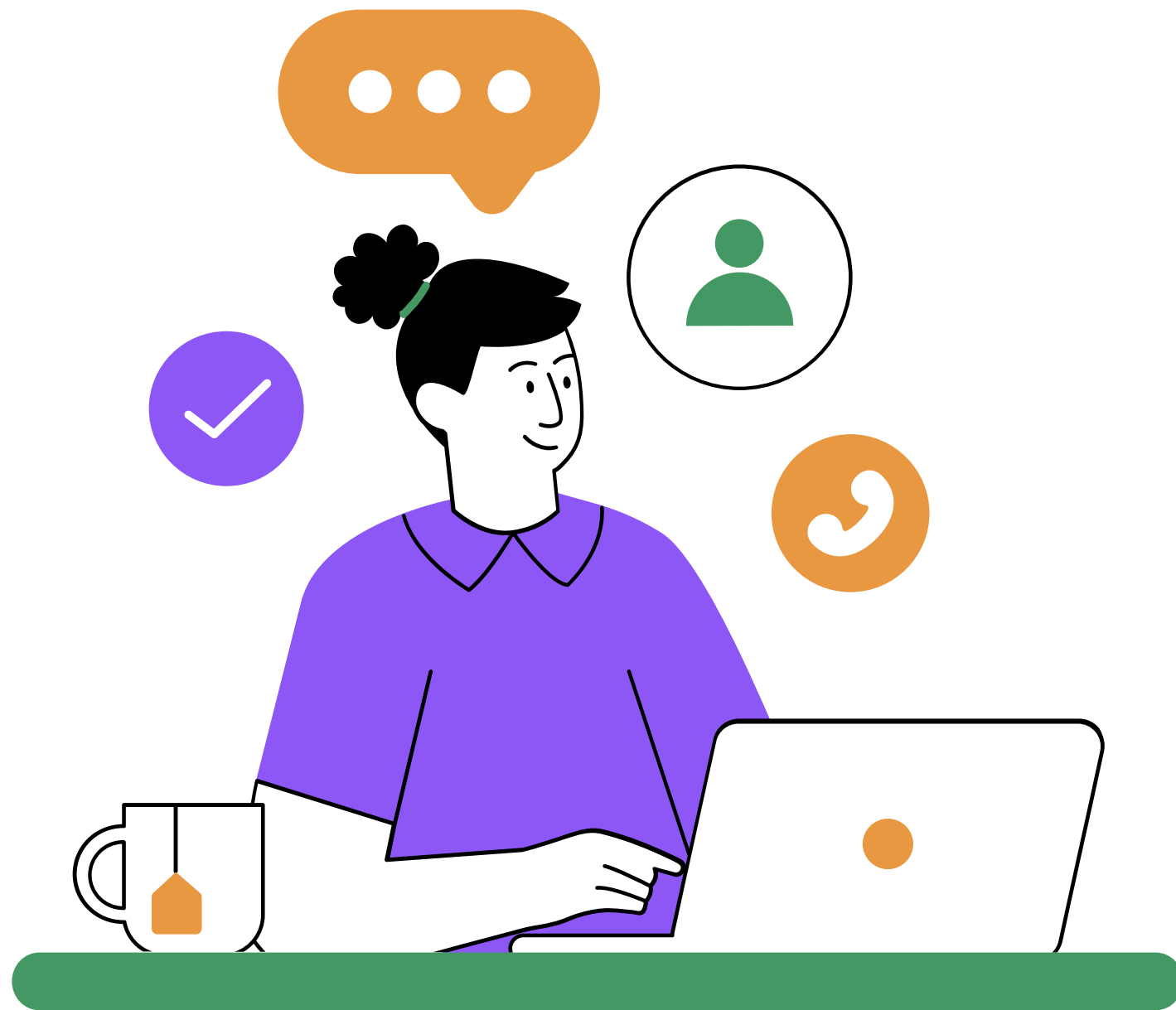
Problem Statements

- 1) Categorize restaurants based on the number of menu items they offer into Small, Medium, and Large restaurants.
- 2) Identify the total number of orders placed by each customer and rank customers according to their ordering activity.
- 3) Find the Top 3 restaurants generating the highest revenue from customer orders.
- 4) Retrieve all customer orders placed within the most recent seven days for recent sales analysis.
- 5) Calculate the total amount spent by every customer across all their food orders.
- 6) Classify customers into Early Bird, Regular, and New categories based on their signup date.
- 7) Identify customers who have placed the maximum number of food orders.
- 8) Find all menu items whose prices are higher than the overall average menu price.
- 9) Identify restaurants that offer more menu items than the average restaurant.
- 10) Generate a monthly order summary and identify months having more than 50 customer orders.



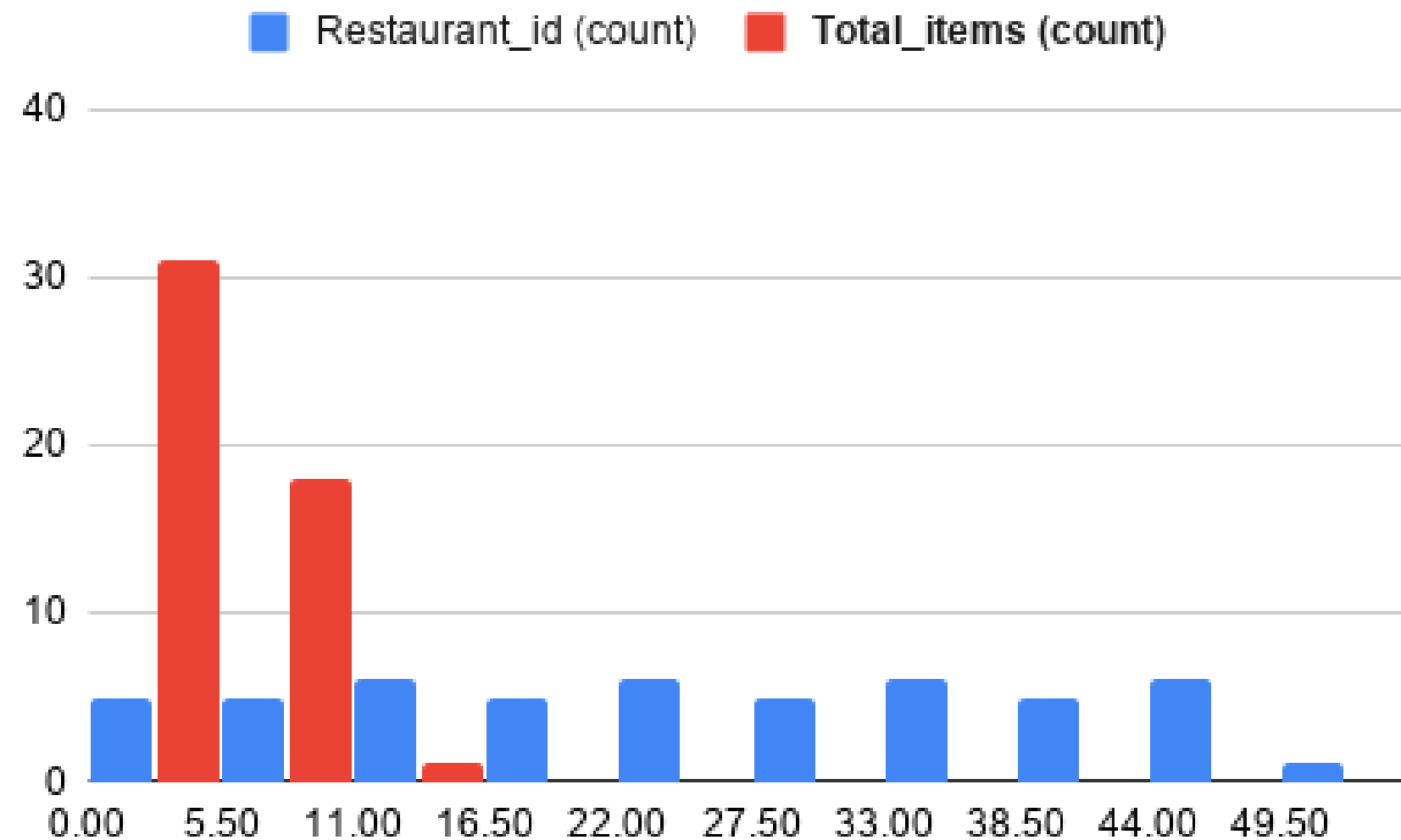
Dataset & Schema

The Online Food Delivery database simulates the operations of a food delivery platform. It stores information about customers, restaurants, menu items, customer orders, and ordered food items. The dataset is used to analyze restaurant performance, customer behavior, menu pricing, and sales trends.



Queries & Methods

Q1: Restaurant Size Categorization

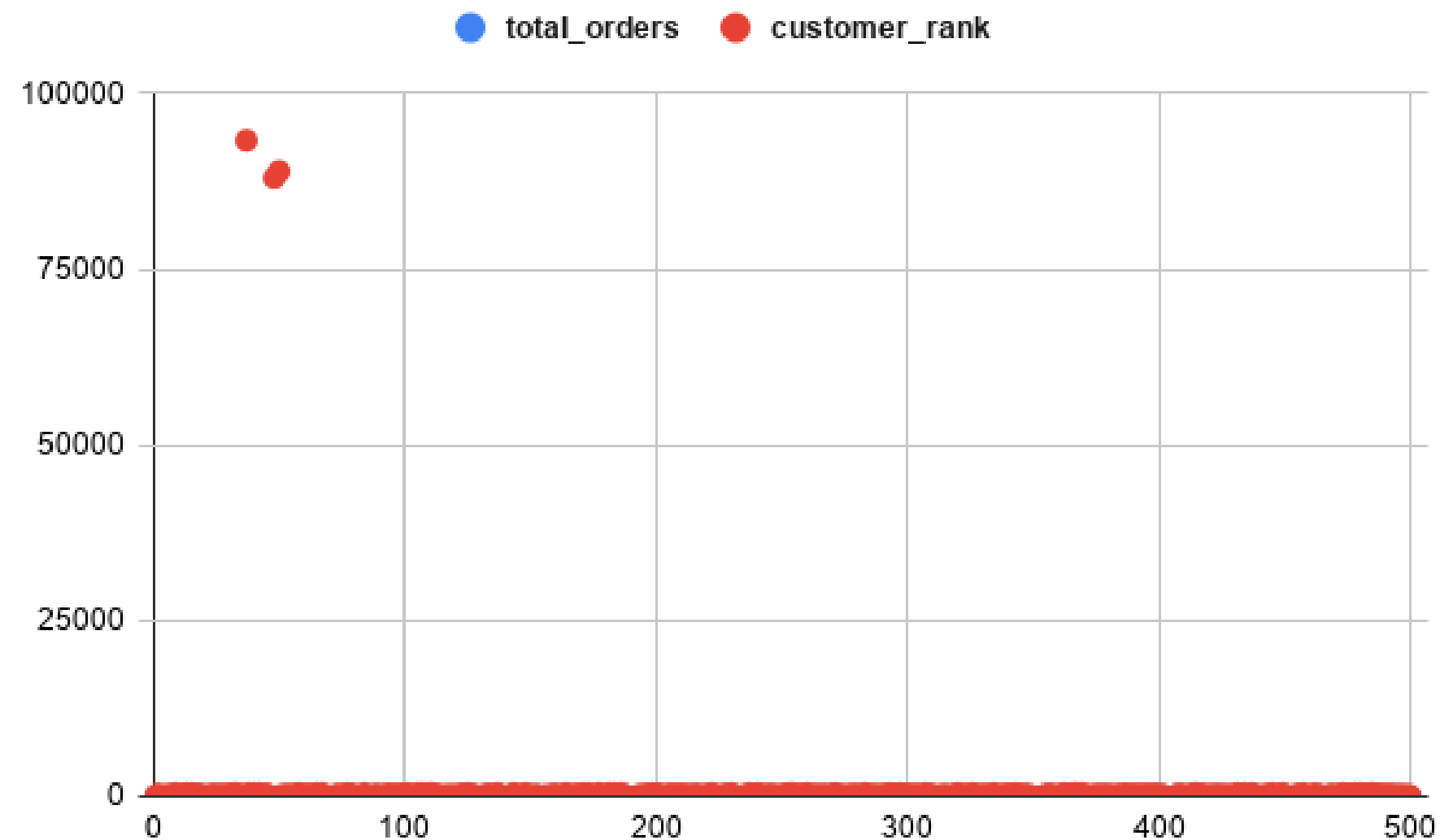


```
SELECT r.restaurant_id, r.rest_name,  
       COUNT(m.item_id) AS total_items,  
       CASE  
         WHEN COUNT(m.item_id) < 5 THEN 'Small'  
         WHEN COUNT(m.item_id) BETWEEN 5 AND 10 THEN 'Medium'  
         ELSE 'Large'  
       END AS restaurant_size  
FROM restaurant r  
JOIN menu_item m  
ON r.restaurant_id = m.restaurant_id  
GROUP BY r.restaurant_id, r.rest_name;
```

Result Grid  Filter Rows: Export: 				
	restaurant_id	rest_name	total_items	restaurant_size
▶	1	Fresh Hub	5	Medium
	2	Big Diner	3	Small
	3	Big Corner	5	Medium
	4	Royal Hub	2	Small
	5	Happy Garden	4	Small
	6	Tasty Bistro	6	Medium
	7	Big Table	5	Medium
	8	Royal Garden	4	Small
	9	Fresh Palace	4	Small
	10	Flavors Corner	6	Medium
	11	Golden Bistro	3	Small
	12	Royal Garden	2	Small

Queries & Methods

Q2: Customer Order Ranking👤

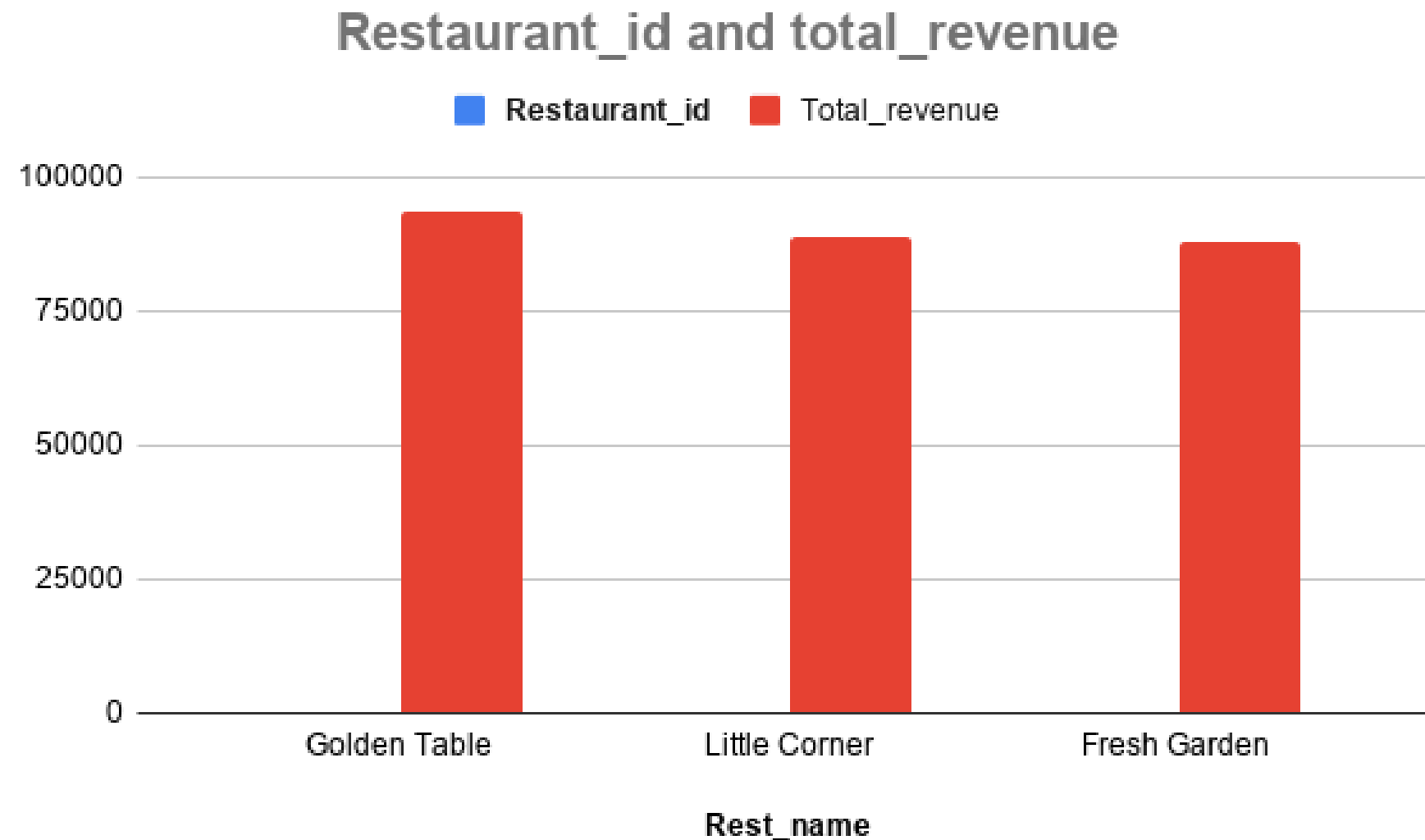


```
WITH customer_orders AS
(
  SELECT customer_id,
         COUNT(order_id) AS total_orders
  FROM orders
  GROUP BY customer_id
)
SELECT customer_id, total_orders,
       RANK() OVER
       (ORDER BY total_orders DESC) AS customer_rank
FROM customer_orders;
```

Result Grid 				Filter Rows:
	customer_id	total_orders	customer_rank	
▶	336	10	1	
	28	9	2	
	154	8	3	
	176	8	3	
	7	7	5	
	72	7	5	
	120	7	5	
	135	7	5	

Queries & Methods

Q3: Top Revenue Generating Restaurants 💰



```
DROP TEMPORARY TABLE IF EXISTS top3_restaurants;  
CREATE TEMPORARY TABLE top3_restaurants AS  
SELECT r.restaurant_id, r.rest_name,  
       SUM(m.price * od.quantity) AS total_revenue  
FROM restaurant r  
JOIN orders o  
ON r.restaurant_id = o.restaurant_id  
JOIN order_details od  
ON o.order_id = od.order_id  
JOIN menu_item m  
ON od.item_id = m.item_id  
GROUP BY r.restaurant_id, r.rest_name  
ORDER BY total_revenue DESC  
LIMIT 3;  
SELECT * FROM top3_restaurants;
```

Result Grid			
Filter Rows:			
	restaurant_id	rest_name	total_revenue
▶	37	Golden Table	93306.66
	50	Little Corner	88911.15
	48	Fresh Garden	87966.50

Queries & Methods

Q4: Store Orders from Last 7 Days July 17

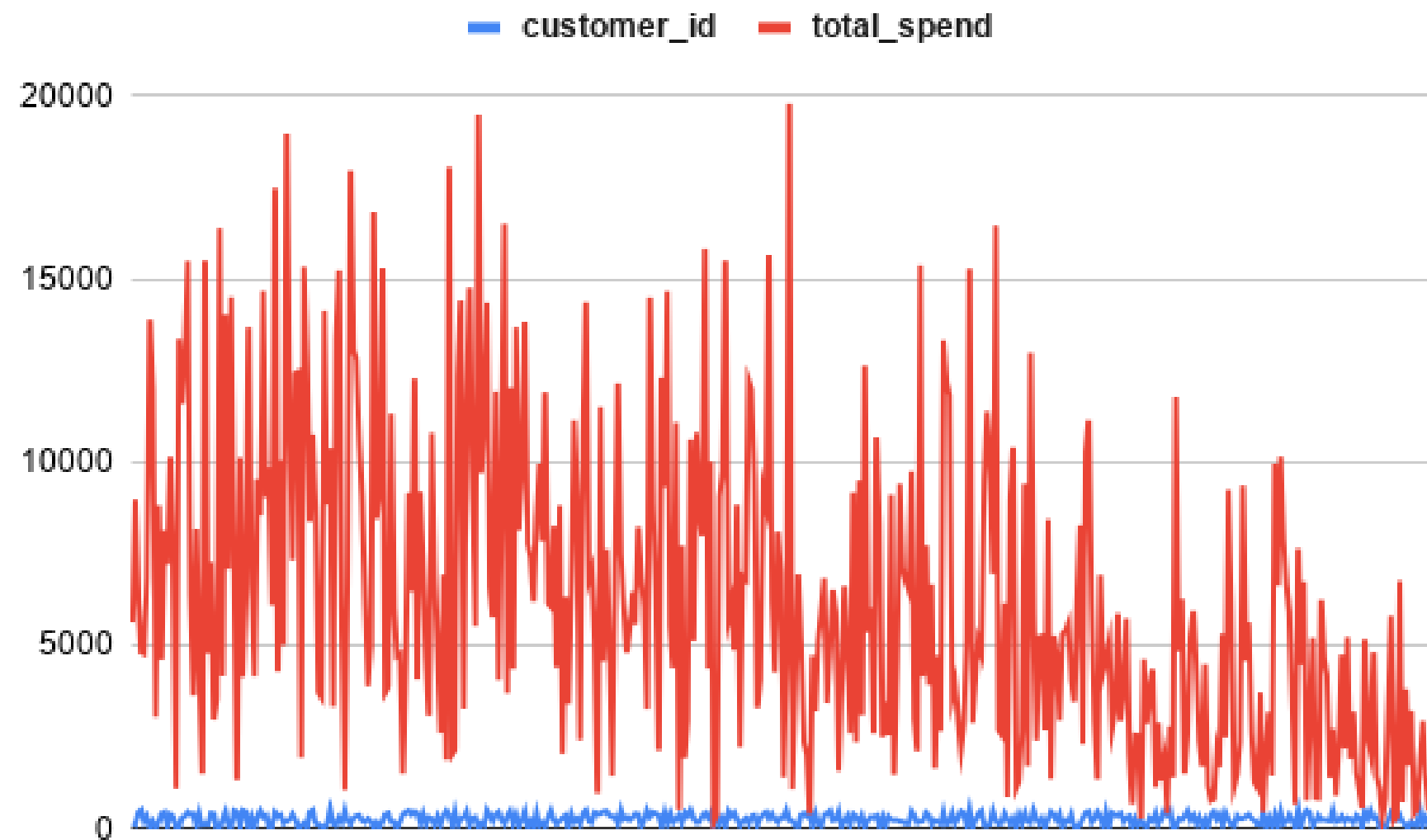


```
DROP TEMPORARY TABLE IF EXISTS last7days_orders;  
CREATE TEMPORARY TABLE last7days_orders AS  
SELECT *  
FROM orders  
WHERE order_date >= (  
    SELECT DATE_SUB(MAX(order_date), INTERVAL 7 DAY)  
    FROM orders  
);  
SELECT * FROM last7days_orders;
```

Result Grid					Filter Rows:	Export
	order_id	customer_id	restaurant_id	order_date		
▶	226	37	34	2025-06-29		
	261	138	47	2025-06-26		
	362	256	27	2025-06-22		
	426	86	41	2025-06-25		
	446	176	11	2025-06-28		
	518	20	26	2025-06-29		
	700	215	42	2025-06-26		

Queries & Methods

Q5: Create View for Customer Spend 👤

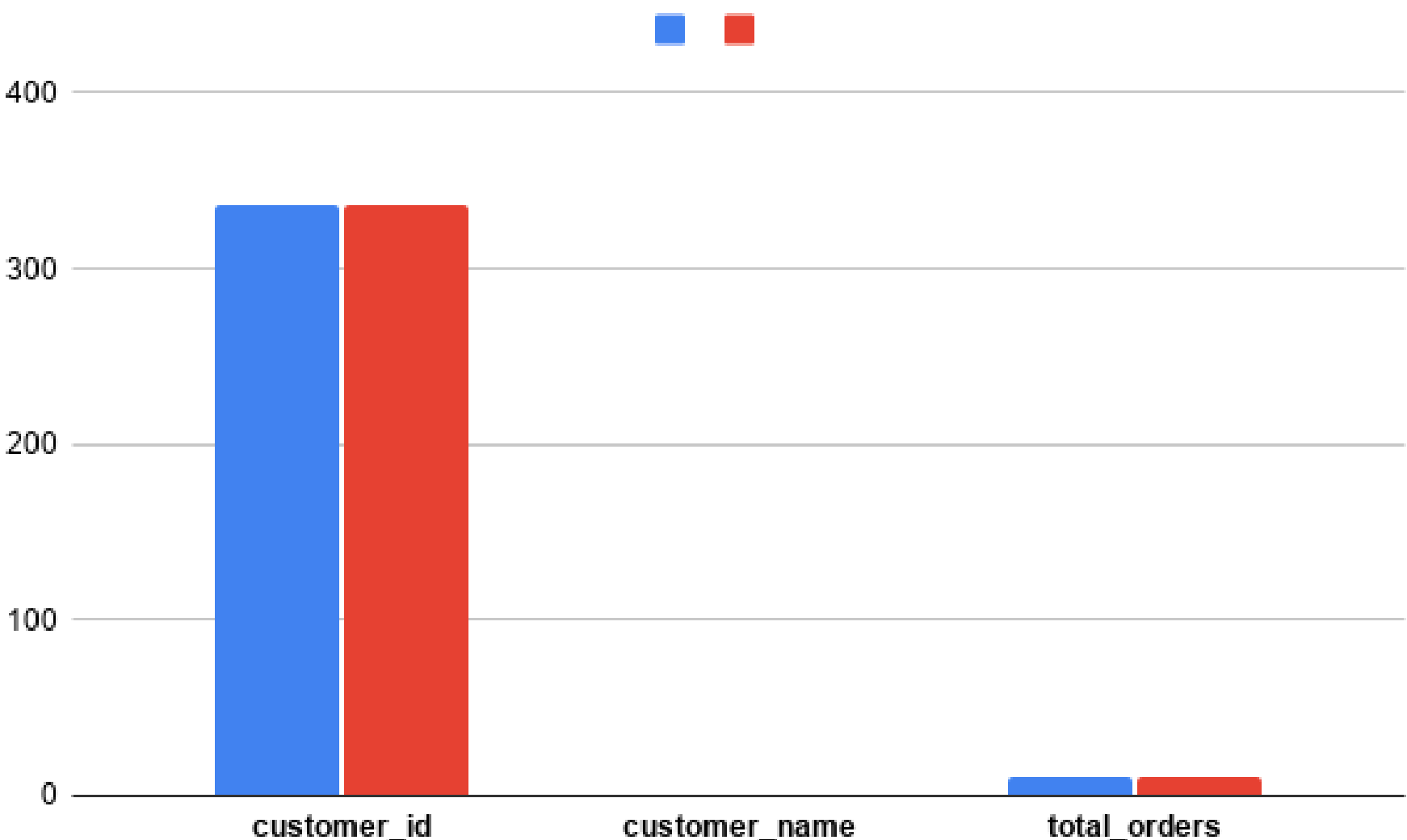


```
CREATE VIEW customer_spend AS
SELECT c.customer_id, c.customer_name,
       SUM(m.price * od.quantity) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_details od ON o.order_id = od.order_id
JOIN menu_item m ON od.item_id = m.item_id
GROUP BY c.customer_id, c.customer_name;
SELECT * FROM customer_spend;
```

Result Grid				Filter Rows:	
	customer_id	customer_name	total_spend		
▶	151	Vihaan Gupta	5617.19		
	104	Muhammad Patel	8980.63		
	418	Arjun Bhat	6288.83		
	497	Aditya Das	4803.14		
	187	Reyansh Sharma	4769.20		
	322	Aarav Bhat	6820.95		
	37	Reyansh Verma	13895.67		
	189	Sai Patel	11983.77		
	1	Vihaan Sharma	2026.14		

Queries & Methods

Q6: Customer Signups Category 👤



```
SELECT customer_id, customer_name, signup_date,  
CASE  
  WHEN signup_date < '2024-01-01'  
  THEN 'Early Bird'  
  WHEN signup_date BETWEEN '2024-01-01'  
  AND '2024-12-31'  
  THEN 'Regular'  
  ELSE 'New'  
END AS customer_category  
FROM customers;
```

Result Grid



Filter Rows:

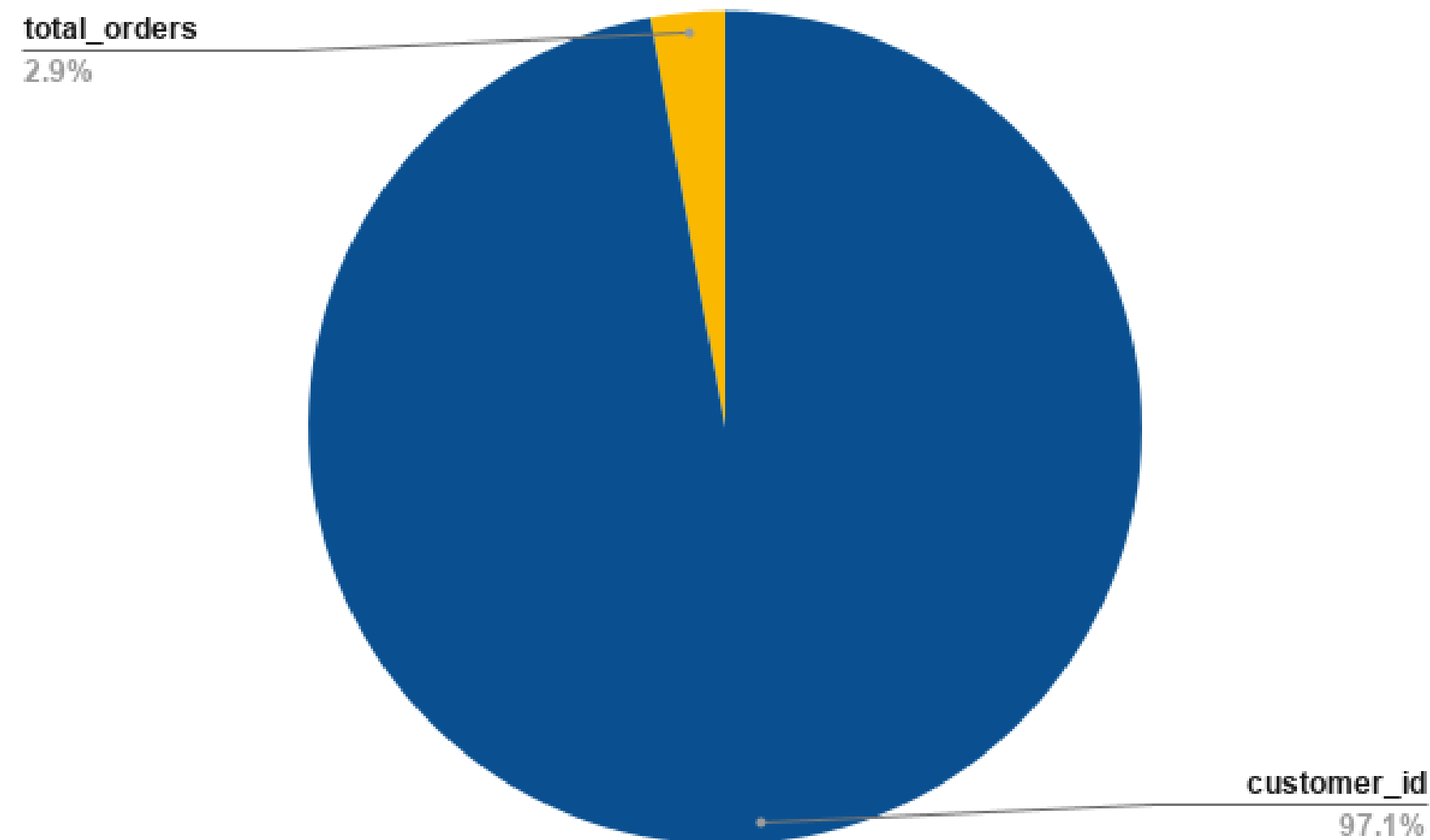


Export:

	customer_id	customer_name	signup_date	customer_category
1		Vivaan Sharma	2023-09-08	Early Bird
2		Vihaan Patel	2024-11-23	Regular
3		Krishna Verma	2024-03-08	Regular
4		Aarav Sharma	2023-08-12	Early Bird
5		Vihaan Das	2023-01-28	Early Bird
6		Krishna Reddy	2024-03-05	Regular
7		Vihaan Nair	2023-10-12	Early Bird
8		Aarav Patel	2023-12-15	Early Bird

Queries & Methods

Q7: Customers with Maximum Orders 👤



```
SELECT c.customer_id, c.customer_name,  
       COUNT(o.order_id) AS total_orders  
FROM customers c  
JOIN orders ON c.customer_id = o.customer_id  
GROUP BY c.customer_id, c.customer_name  
HAVING COUNT(o.order_id) = (  
    SELECT MAX(order_count)  
    FROM (  
        SELECT COUNT(*) AS order_count  
        FROM orders  
        GROUP BY customer_id  
    ) AS t  
);
```

Result Grid		Filter Rows:	Export:
	customer_id	customer_name	total_orders
▶	336	Ishaan Sharma	10

Queries & Methods

Q8: Menu Items Priced Above Global Average

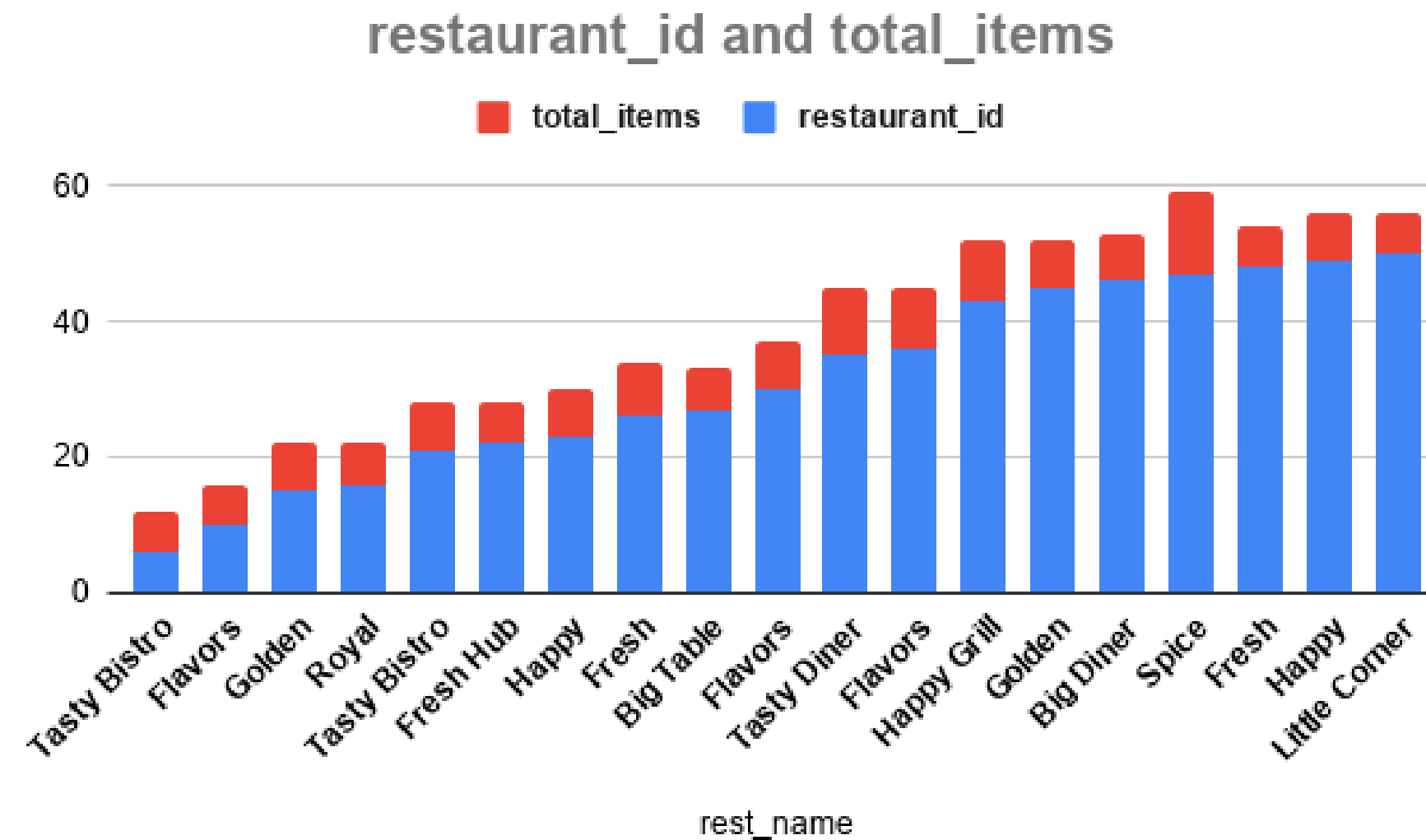


```
SELECT item_id,item_name,price
FROM menu_item
WHERE price >
(
    SELECT AVG(price)
    FROM menu_item
);
```

Result Grid			
Filter Rows:			
	item_id	item_name	price
▶	1	Rasgulla	347.79
	6	Gulab Jamun	565.55
	8	Fish Curry	597.90
	9	Fish Curry	537.12
	12	Hakka Noodles	383.34
	16	Rasgulla	477.21
	20	Chicken 65	477.49
	21	Samosa Chaat	433.81
	22	Masala Dosa	587.04
	23	Rasgulla	493.35

Queries & Methods

Q9: Restaurants With More Items Than Average 💰

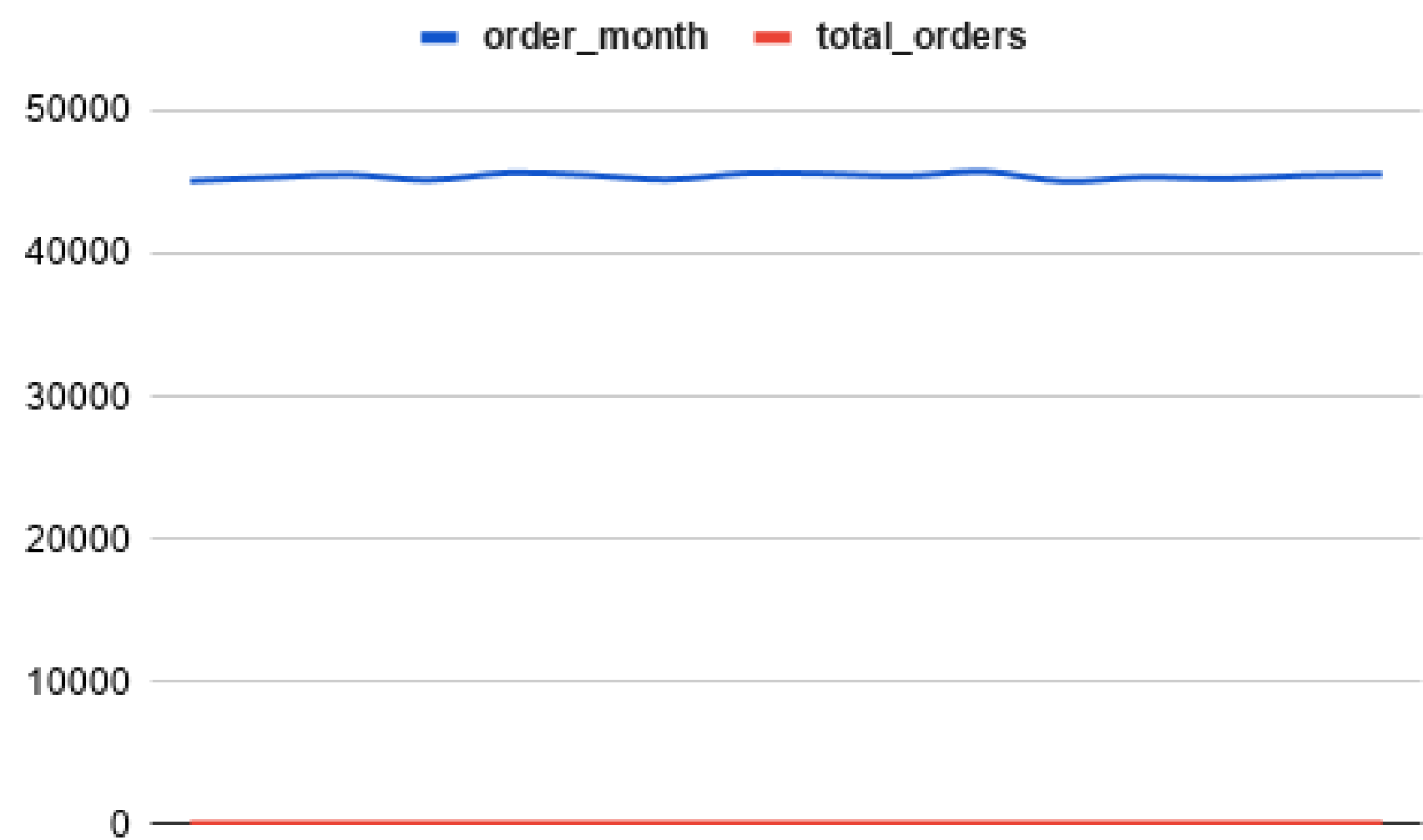


```
SELECT r.restaurant_id, r.rest_name,
       COUNT(m.item_id) AS total_items
FROM restaurant r
JOIN menu_item m ON r.restaurant_id = m.restaurant_id
GROUP BY r.restaurant_id, r.rest_name
HAVING COUNT(m.item_id) >
(
  SELECT AVG(item_count)
  FROM
  (
    SELECT COUNT(*) AS item_count
    FROM menu_item
    GROUP BY restaurant_id
  ) AS avg_items
);
```

Result Grid			
Filter Rows:			
	restaurant_id	rest_name	total_items
▶	6	Tasty Bistro	6
	10	Flavors Corner	6
	15	Golden Garden	7
	16	Royal Palace	6
	21	Tasty Bistro	7
	22	Fresh Hub	6
	23	Happy Corner	7

Queries & Methods

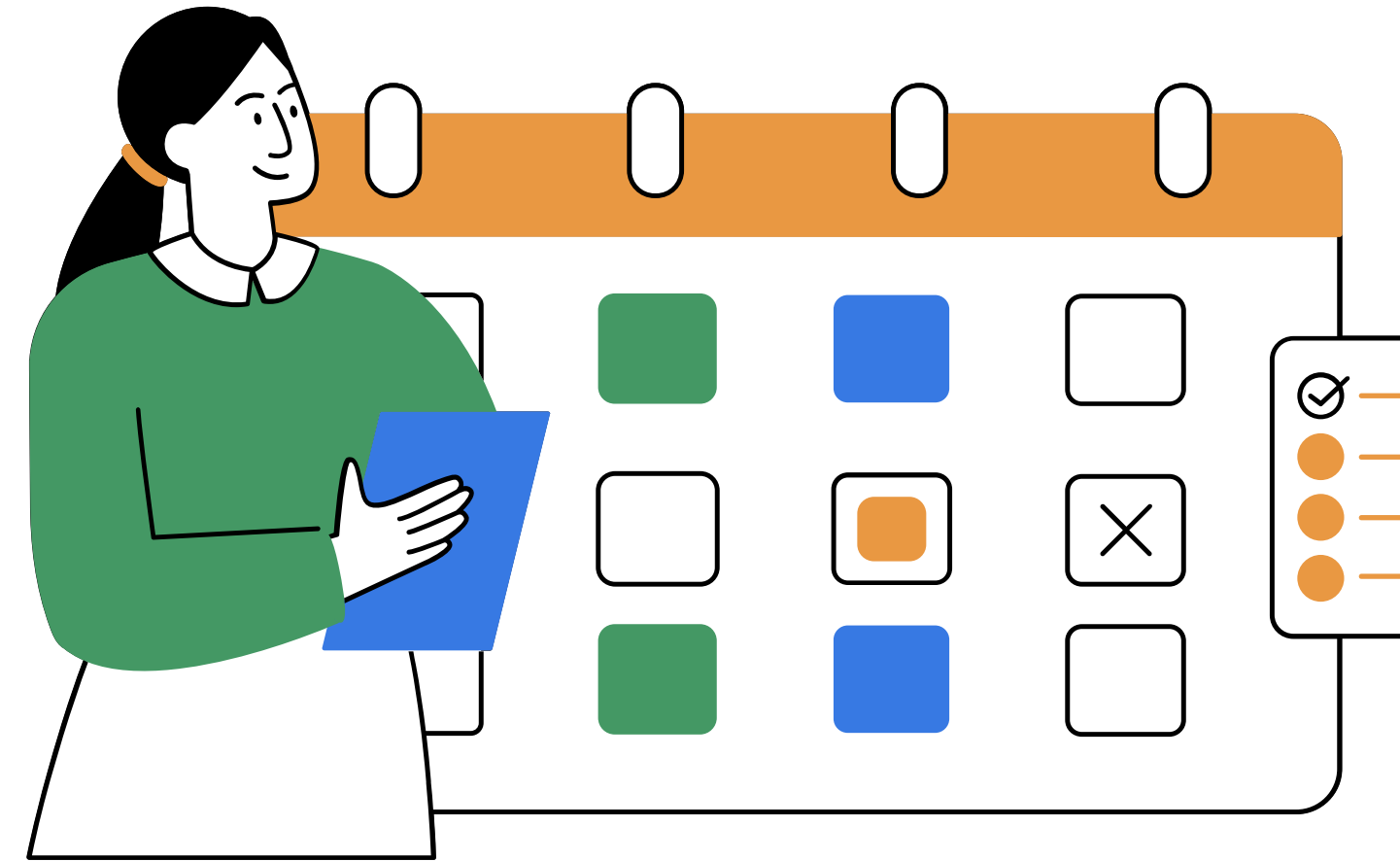
Q10: Monthly Order Summary July 17



```
WITH MonthlyOrders AS
(
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS order_month,
        COUNT(order_id) AS total_orders
    FROM orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT * FROM MonthlyOrders WHERE total_orders > 50;
```

Result Grid			Filter Rows:
	order_month	total_orders	
▶	2023-05	61	
	2024-01	55	
	2024-08	55	
	2023-07	63	
	2024-12	52	
	2024-07	63	
	2023-08	52	
	2024-11	54	

Insights & Storytelling



Point 01

Business Insight :

Restaurants with larger menus generally provide more variety to customers, whereas smaller restaurants focus on specialized cuisine.



Point 02

Business Insight :

Highly active customers can be rewarded through loyalty programs and personalized offers.

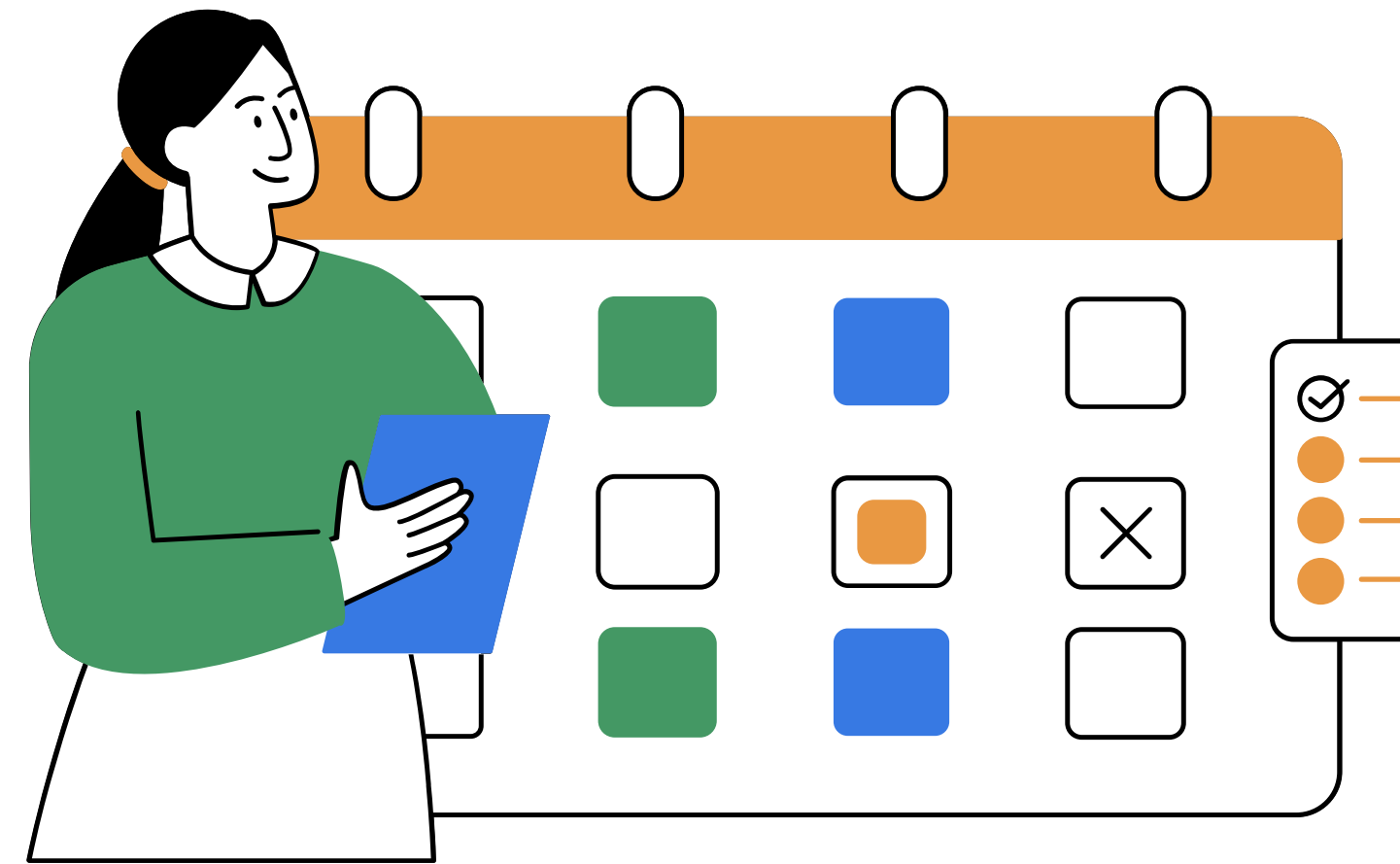


Point 03

Business Insight :

Top-performing restaurants contribute significantly to platform revenue and should receive promotional support.

Insights & Storytelling



Point 04

Business Insight :
Recent orders help analyze
current customer demand and
delivery performance.



Point 05

Business Insight :
Customers with higher spending
can be targeted with premium
memberships and exclusive
discounts.



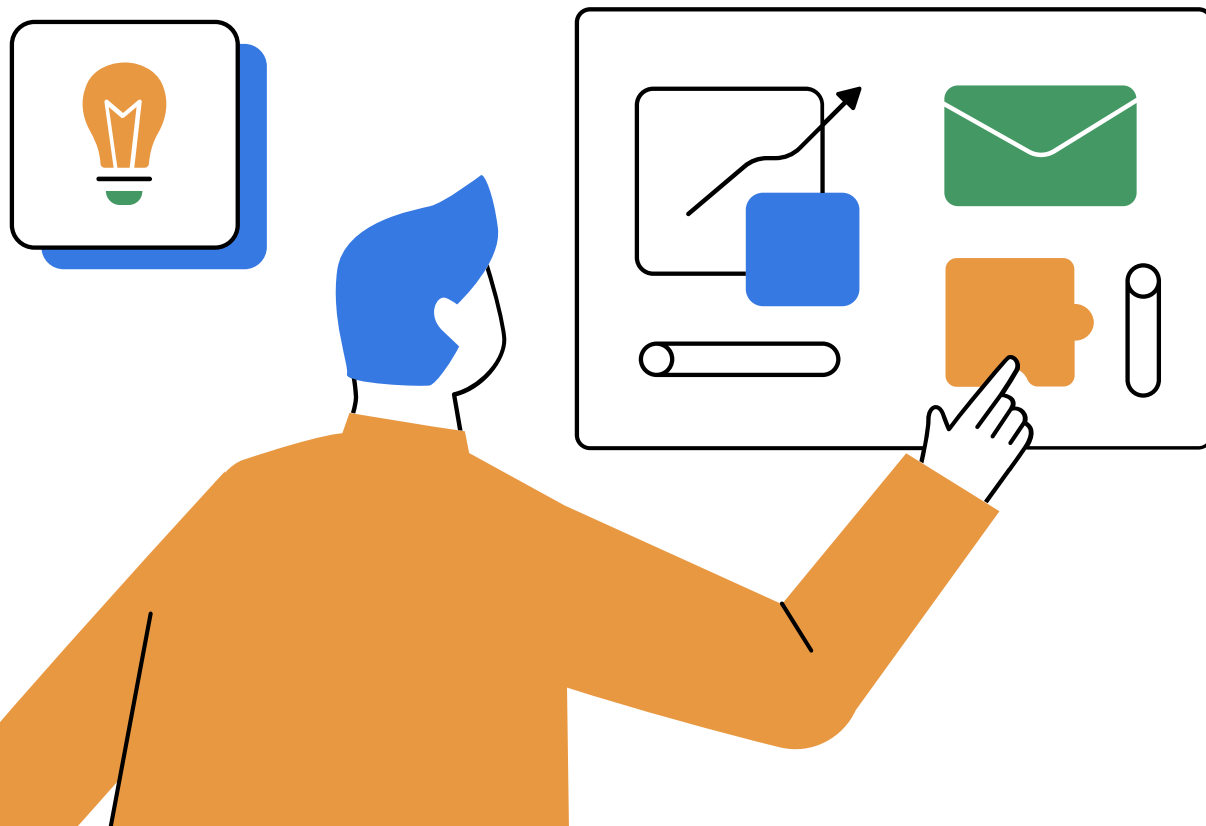
Point 06

Business Insight :
Premium-priced menu items
contribute higher revenue and
help understand pricing strategy.

Conclusion & Recommendations

Conclusion

This project successfully demonstrates how advanced SQL techniques can solve real-world business problems in an Online Food Delivery platform. By analyzing customer behavior, restaurant performance, menu pricing, and sales trends, meaningful business insights were generated that support data-driven decision-making.



Recommendations

- Expand menu options for restaurants with limited food variety.
- Reward loyal customers through membership or loyalty programs.
- Promote top-performing restaurants to maximize revenue.
- Monitor recent order trends for better delivery planning.
- Optimize menu pricing based on customer demand.
- Use monthly sales reports to forecast future business growth.

Thank You

Thank you for your time and attention.

Get in Touch

K.N.V.S. Kiran Kumar

● 9392626389

● kavalakiran98@gmail.com

